

Description de l'implémentation de l'outil d'aide à la décision (OAD) et architecture logicielle

7 juillet 2026

1 Introduction

Notre outil d'aide à la décision pour la planification est développé via `pygame` (`pip install pygame`). Il y a 3 onglets principaux sur l'interface : la planification, la simulation, et la supervision.

2 Orchestration Globale : `main.py`

Le fichier `main.py` est le chef d'orchestre de l'application.

- Initialisation et configuration de l'environnement
- Boucle principale de jeu (`while self.running:`)
- Aiguillage des événements (`handle_events()`)
- Affichage

3 Les 3 onglets

3.1 La planification (`planning_tab.py`)

Cet onglet est le plus vaste de l'application. Il contient le pattern Modèle-Vue-Contrôleur en un seul fichier. L'utilisateur se sert de cet onglet pour planifier le projet en amont.

A. Les data classes

Ces classes stockent l'info courante du planning. Elles transitent dans toute l'app par référence.

- **Task** : représente une tâche de production. Contient l'ID, le nom, l'activité, la durée (`duration`), l'équipe d'opérateur requise (`pool_name`), l'opérateur assigné (`assigned_to`), le moment de début (`week_start`), le retard accumulé (`delay_duration`), et l'index de découpage (`part_index` pour les tâches en plusieurs parties).
- **Operator** : Représente un opérateur. Contient un ID, un nom, un `pool_name` (ex : "Assembleur"), une liste d'`absences`, et surtout une liste `tasks` contenant les références vers les objets `Task` qui lui sont assignés.

B. Les composants visuels

Ces classes gèrent l'affichage et les clics, elles fonctionnent avec des pointeurs vers les objets.

- **TaskBlock** : Représente visuellement une tâche déjà planifiée sur le diagramme de Gantt, avec ses aléas.

- **TaskCard** : Représente visuellement une tâche non planifiée (en attente dans le panier).
- **ActivityBox** : Un bouton/dossier représentant un groupe de tâches de la même activité.
- **Timeline** : La zone principale du Gantt. Mode *détaillé* ou mode *agrégé*.
- **TaskBasket** : Conteneur visuel qui englobe les **ActivityBox** et les **TaskCard**.

C. Le Contrôleur Principal : PlanningTab

Il possède les listes maîtresses `self.tasks` et `self.operators`.

- **Drag & Drop** : Lorsqu'un **TaskBlock** est relâché sur la **Timeline**, **PlanningTab** calcule la collision. Si valide, il retire la référence **Task** de la liste `tasks` de l'ancien **Operator**, l'ajoute au nouvel **Operator**, et met à jour `week_start`.
- **Solveur** : Lors de l'appel à une fonctionnalité du solveur (*Nouvelle solution*, *Placer sans conflit*, *Placer au plus tôt*), **PlanningTab** injecte `self.tasks` et `self.operators` dans le **PlanningSolver**. Le solveur modifie directement les objets en mémoire.
- **Undo/Redo** : **PlanningTab** maintient un historique (`self.history`). À chaque action, il fait une deepcopy des listes `tasks` et `operators` pour sauvegarder l'état du planning.

3.2 La simulation de solutions (simulation_tab.py)

Cet onglet permet à l'utilisateur de collaborer de façon haut-niveau avec le solveur.

- **RadarWidget** : Un composant visuel complexe gérant un radar où l'utilisateur peut imposer des contraintes spécifiques au solveur.

3.3 La supervision de planning (monitoring_tab.py)

Dans cet onglet, l'utilisateur supervise le déroulement en direct de sa planification. Quand des aléas arrivent, l'utilisateur est renvoyé vers l'onglet de planification, où il doit faire des modifications pour réparer le planning. Pour l'instant, les plannings et les aléas sont hard-codés.

A. Flux de Données

- **Modification en temps réel** : Les aléas de **MonitoringTab** modifient directement les attributs des objets **Task** ciblés.
- **Synchronisation du temps courant** : Quand l'utilisateur doit aller replanifier, c'est exactement comme la planification de base, mis à part que toutes les tâches passées ne sont pas modifiables.

4 Utilitaires généraux

- `ui_components.py` : classes **Button** et **HorizontalScrollbar**.
- `solver_cpo.py` : Fournit la classe **PlanningSolver**. Elle prend en entrée les listes d'objets **Task** et **Operator**, les convertit en variables CPO (modèle CPLEX), lance la résolution, puis "dépile" le résultat CPLEX pour **réécrire** les attributs `week_start` et `assigned_to` des objets Python originaux.